

ICP133 - Fundamentos de Sistemas de Computação

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 14, 2026

Aula 1 - Sistemas analógicos e digitais

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 14, 2026

O Sistema do Telégrafo

- **Funcionamento Eletromecânico:**

- Circuito composto por bateria, chave (transmissor) e bobina (receptor).
- O fechamento da chave cria um campo magnético que atrai uma placa metálica ("estalido").
- O retorno por mola ao abrir o circuito gera um som distinto.

- **Representação Digital:**

- Uso de dois símbolos distintos: **pontos** (pulsos curtos) e **traços** (pulsos longos).
- Base do Código Morse: a informação é codificada na **duração** e na **sequência** dos pulsos.

Diagramas de Tempos e Sinais

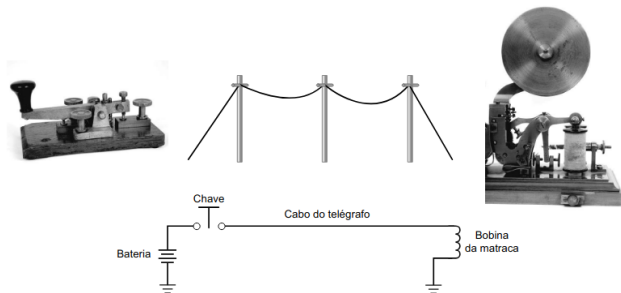


FIGURA 1.1 Sistema do telégrafo.

- **Natureza do Sinal:**

- O sinal elétrico é binário: está sempre **ligado** ou **desligado**.
- Precursor dos sistemas modernos (níveis lógicos 0 e 1).

- **Diagrama de Tempos:**

- Gráfico da tensão (ou corrente) versus tempo ($x = t$).
- **Função:** Mostra o estado do sistema em qualquer instante e o momento exato das transições.

Diagramas de Tempo

- Representação gráfica da atividade em sistemas digitais
- Eixo x: tempo
- Eixo y: estado (1 ou 0)
- Mostra:
 - Estado do sistema em qualquer momento
 - Momento exato das mudanças de estado
- **Ferramenta fundamental** para análise de sistemas digitais

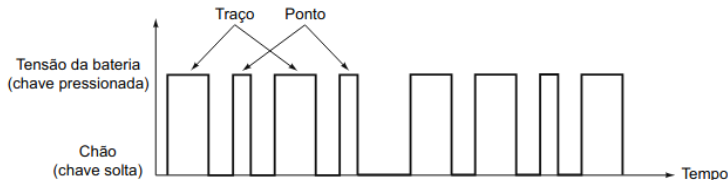


FIGURA 1.2 Diagrama de tempos de uma linha de telégrafo.

- **Definição:** Uma quantidade é representada por um indicador proporcional **continuamente variável**.
- **Características Principais:**
 - Acompanha variações mínimas de forma fluida.
 - O indicador (ponteiro, coluna, tensão) "segue" a grandeza física.
- **Exemplos Clássicos:**
 - **Velocímetro Mecânico:** Deflexão do ponteiro proporcional à velocidade do eixo.
 - **Termômetro de Mercúrio:** Altura da coluna varia com a temperatura.
 - **Termômetro de Mola:** Expansão/contração térmica converte-se em posição angular.

- **Conversão de Grandezas:**

- A quantidade física original é convertida em um **signal elétrico**.
- A **tensão** ou **corrente** resultante é diretamente proporcional à medida.

- **Aplicações:**

- Exibição em escalas graduadas.
- Processamento de sinais em tempo real.
- Sistemas de controle contínuo.

O Som como Grandeza Analógica

- **Histórico:** Alexander Graham Bell (1875) converteu voz em sinal elétrico continuamente variável.
- **O Microfone:** Atua como um transdutor que converte energia sonora em **tensão analógica**.

Onda de Audio

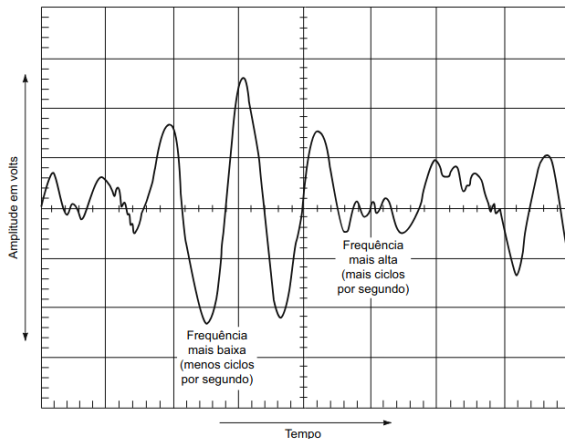


FIGURA 1.4 Onda de áudio.

Os valores variam ao longo de uma **faixa contínua**

- **Definição:** Quantidades representadas por símbolos chamados **dígitos**, em vez de indicadores contínuos.
- **Funcionamento:**
 - A variação ocorre em **incrementos discretos** (saltos).
 - Exemplo: Um termômetro digital que mede de $0,1^{\circ}\text{C}$ em $0,1^{\circ}\text{C}$.
 - A temperatura real sobe de forma contínua, mas o visor "pula" de $22,0^{\circ}\text{C}$ para $22,1^{\circ}\text{C}$.
- **Precisão e Resolução:**
 - No digital, a precisão depende do número de dígitos ou "passos" disponíveis no sistema.

Analógico vs. Digital

Diferença Fundamental

Analógica – > **Contínua**

Digital – > **Discreta**

Ou seja....

Diferente do digital (que sobe em "degraus"), o analógico não possui "saltos em degraus"; entre dois valores quaisquer, existe uma infinidade de valores intermediários.

- **Interpretação Analógica:**

- Frequentemente sujeita a "interpretação livre"
 - ex: posição exata de um ponteiro entre dois traços
- Depende da percepção do observador.

- **Leitura Digital:**

- A leitura é direta e numérica.
- Não há dúvida sobre o valor exibido
 - ex: 22,5°C é lido exatamente

- **O Processo de Digitalização:**

- Na prática, sempre "arredondamos" medidas analógicas para um nível de precisão conveniente.
- **Digitalizar:** Atribuir um número de **precisão limitada** a uma quantidade continuamente variável.

- **Captura do Mundo Real:**

- O mundo é analógico (variáveis mudam constantemente).
- Ao medir e converter essas variáveis em quantidades digitais, ganhamos novas possibilidades.

- **Vantagens do Processamento Digital:**

- **Registro:** Armazenamento fiel de dados ao longo do tempo.
- **Manipulação:** Cálculos aritméticos e algoritmos complexos.
- **Controle:** Uso das informações para acionar ou gerenciar sistemas.

Quiz - Analógico ou digital?

Exemplo 1.1

Dentre as quantidades a seguir, quais estão relacionadas a quantidades analógicas e quais estão relacionadas a quantidades digitais?

- (a) Subida usando uma escada.
- (b) Subida usando uma rampa.
- (c) Corrente que flui de uma tomada elétrica por meio um motor.
- (d) Altura de uma criança medida por uma fita métrica em divisão de 1 cm.
- (e) Altura de uma criança colocando uma marca na parede.
- (f) Volume de areia em um balde.
- (g) Volume de água em um balde.

Solução

- (a) Digital.
- (b) Analógica.
- (c) Analógica.
- (d) Digital: medido até o meio centímetro mais próximo.
- (e) Analógica.
- (f) Digital: só pode aumentar/diminuir por grãos discretos de areia.
- (g) Analógica: (a não ser que você queira chegar ao nível da nanotecnologia!).

- **Sistemas Digitais:**

- Combinação de dispositivos para manipular informação lógica ou quantidades em **valores discretos**.
- **Meios:** Eletrônicos (mais comuns), mecânicos, magnéticos ou pneumáticos.
- **Exemplos:** Computadores, calculadoras, sistemas de telecomunicações.

- **Sistemas Analógicos:**

- Dispositivos que manipulam quantidades em **faixas contínuas**.
- **Exemplo:** O sinal de saída de um alto-falante (pode assumir qualquer amplitude entre zero e o máximo).

- **Integração:** É comum o uso de analógico e digital mesmo projeto para aproveitar as vantagens específicas de cada uma.
- **Desafio de Projeto:** Determinar quais partes do fluxo devem ser analógicas e quais devem ser digitais.
- **Tendência Atual:**
 1. Digitalizar o sinal **o mais cedo possível**.
 2. Processar digitalmente.
 3. Converter de volta para analógico **o mais tarde possível**.

A digitalização precoce preserva a integridade da informação contra ruídos e facilita o processamento complexo.

Vantagens das Técnicas Digitais (1/2)

- **Facilidade de Projeto:**

- Baseado em circuitos de **chaveamento**.
- Importam apenas as faixas (*HIGH* ou *LOW*), não valores exatos de tensão.

- **Armazenamento Eficiente:**

- Dispositivos especiais (*latches*) mantêm a informação indefinidamente.
- Armazenamento de massa: bilhões de bits em espaços reduzidos.

- **Precisão e Exatidão:**

- A informação não se deteriora durante o processamento.
- Sistemas analógicos sofrem com variações de temperatura, umidade e tolerância de componentes.

Vantagens das Técnicas Digitais (2/2)

- **Operações Programáveis:**

- Comportamento controlado por conjuntos de instruções (programas).
- Flexibilidade e complexidade muito superiores aos sistemas analógicos.

- **Imunidade ao Ruído:**

- Flutuações de tensão (ruído) são desprezíveis, desde que não alterem a distinção entre nível **ALTO** e **BAIXO**.

- **Alta Integração (CIs):**

- Chips digitais comportam milhões de dispositivos.
- Analógicos exigem componentes difíceis de integrar (grandes capacitores, indutores, resistores de precisão).

Limitações das Técnicas Digitais

Apesar das vantagens, existem dois desafios principais na adoção do digital:

- **O Mundo Real é Analógico:**
 - Grandezas físicas (temperatura, pressão, velocidade, vazão) são contínuas por natureza.
 - Representações como "18°C" são apenas **aproximações digitais** da realidade.
- **Tempo de Processamento:**
 - Converter, processar e desconverter sinais exige tempo (latência), o que pode ser crítico em sistemas de altíssima velocidade.

Exemplos de Sensores em um Automóvel

Nível de combustível, temperatura do motor, acelerômetros (airbag) e sensores de pressão.

Para integrar o processamento digital ao mundo físico, seguimos quatro passos fundamentais:

1. **Transdução:** Converter a variável física (ex: calor) em sinal elétrico analógico.
2. **Conversão A/D:** Transformar o sinal analógico em formato digital (bits).
3. **Processamento:** Operar a informação digitalmente (cálculos, lógica).
4. **Conversão D/A:** Converter o resultado digital de volta para o formato analógico do mundo real.

- **Entrada de Usuário (Digital):** Ajuste da temperatura desejada em incrementos discretos (ex: 0, 1°C).
- **Captura (Analógica):** Um sensor converte a temperatura ambiente em uma **tensão proporcional**.
- **Interface de Processamento:**
 - **ADC (Conversor Analógico-Digital):** Traduz a tensão do sensor para um valor binário.
 - **Processador:** Compara o valor real com o desejado e calcula a correção necessária.

- **Saída do Sistema:**

- **DAC (Conversor Digital-Analógico):** Converte o ajuste digital calculado de volta para uma **tensão analógica**.
- **Atuador:** O dispositivo de aquecimento recebe essa tensão e gera calor proporcional para o ambiente.

- **Resumo do Fluxo:**

Temp. Física → Sensor → ADC → Lógica Digital → DAC → Aquecedor

Importância da Conversão

Este ciclo permite que a precisão da lógica digital controle variáveis instáveis e contínuas do mundo físico.

Exemplo

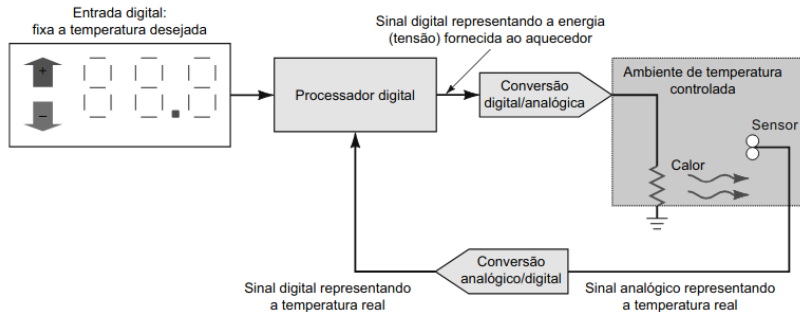


FIGURA 1.7 Diagrama de um sistema de controle de temperatura de precisão que utiliza processamento digital.

Exemplo de Transdutor: O LDR

- **Definição:** O *Light Dependent Resistor* (Resistor Dependente de Luz) é um componente analógico cuja resistência varia com a intensidade luminosa.
- **Comportamento Analógico:**
 - Varia a resistência com a luz.
 - A variação é **contínua** e proporcional à luz recebida.
- **Integração Digital:**
 - O LDR envia uma **tensão analógica** para um pino do microcontrolador.
 - O **ADC** interno converte essa tensão em um número.

Exemplo Prático: Leitura Analógica (Arduino)

- Captura de um sinal analógico e sua conversão digital via **ADC**.
- A função `analogRead()` transforma a tensão no pino A0 em um valor digital.

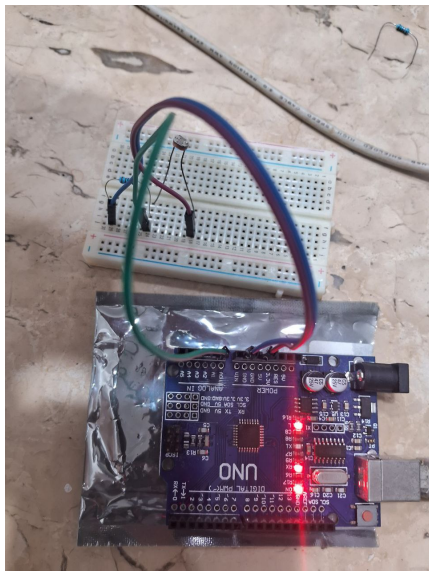
Código de Leitura do LDR

```
int valorldr = 0;

void setup() {
  pinMode(A0, INPUT);
  Serial.begin(9600);
}

void loop() {
  valorldr=analogRead(A0);
  Serial.print("Valor lido = ");
  Serial.println(valorldr);
}
```

Leitor de Luminosidade



Função `analogRead()`

- **Descrição:** Lê o valor de um pino de entrada analógica especificado.
- **Conversor Analógico-Digital (ADC):**
 - O Arduino UNO possui um ADC de **10 bits**.
 - Mapeia tensões entre 0 e 5 V em valores inteiros entre **0 e 1023**.
- **Resolução do Sistema:**
 - Cálculo: $\frac{5\text{ V}}{1024 \text{ unidades}} \approx 0,0049 \text{ V (4,9 mV)}$ por unidade.

Observações Técnicas

- **`analogReference()`:** Altera a faixa de tensão de entrada.
- **`analogReadResolution()`:** Permite alterar a resolução em placas que suportam mais de 10 bits.

Link da API de referência

- **Uma cadeia:**

1. **Fenômeno Físico:** Intensidade de luz varia.
2. **Efeito Analógico:** A resistência do LDR muda, alterando a **tensão (V)** no divisor de tensão.
3. **Processamento:** O microcontrolador lê essa tensão contínua.
4. **Interpretação:** O software associa o valor digital à condição de "escuro".

Nesse caso medimos a luz indiretamente através da tradução de grandezas elétricas

- **Contexto:** Na aula passada, vimos que sistemas digitais operam com valores **discretos**.
- **A Necessidade de Símbolos:** Para representar informações digitalmente, precisamos de dígitos.
- **Por que apenas dois dígitos?**
 - É mais simples e confiável construir dispositivos eletrônicos que reconheçam apenas dois estados:
 - **Aberto** vs. **Fechado**
 - **LOW** (Baixo) vs. **HIGH** (Alto)
 - **0** vs. **1**

Sistema Binário

Usa a base 2 para converter esses estados físicos em informação matemática.

Aula 2 - Sistemas de Numeração Digital

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 14, 2026

- **Panorama Geral:** Na tecnologia digital, utilizamos diferentes bases numéricas para finalidades específicas.
- **Bases mais usadas:**
 - Decimal (Base 10)
 - Binário (Base 2)
 - Octal (Base 8)
 - Hexadecimal (Base 16)

Sistema Decimal (Base 10)

- **Símbolos:** Composto por 10 dígitos: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.
- **Origem:** Evoluiu do uso dos dez dedos das mãos para contagem
- *dígito* vem do latim *digitus*, "dedo".
- **Sistema Posicional:** O valor de um dígito depende da sua posição no número.

Contagem decimal

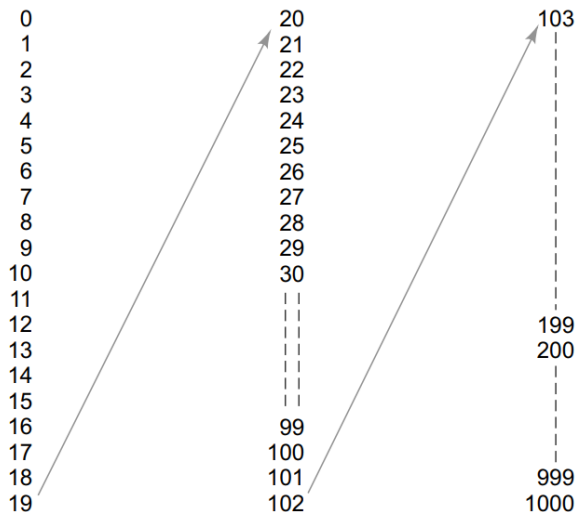


FIGURA 1.9 Contagem decimal.

Sistema Decimal (Base 10)

Exemplo: Número 453

4	5	3
Centenas (10^2)	Dezenas (10^1)	Unidades (10^0)

- **MSD (Most Significant Digit):** O dígito de maior peso (neste caso, o 4).
- **LSD (Least Significant Digit):** O dígito de menor peso (neste caso, o 3).

Representação Formal do Sistema Decimal

- **Estrutura Genérica:** Qualquer número decimal pode ser escrito como uma sequência de dígitos d em relação à vírgula:

$$d_n d_{n-1} \dots d_1 d_0, d_{-1} d_{-2} \dots d_{-k}$$

Expansão Polinomial

O valor real é a soma de cada dígito multiplicado pelo peso de sua posição:

$$Valor = \sum_{i=-k}^n d_i \times 10^i$$

- **Componentes:**
 - d_i : O algarismo na posição i (de 0 a 9).
 - 10^i : O peso (base elevada à posição).
 - n e k : Número de dígitos inteiros e fracionários, respectivamente.

Essa mesma lógica será aplicada ao sistema binário, apenas trocando a base 10 por 2.

Formalização da Notação Posicional

- **Definição:** A posição do dígito determina a potência da base pela qual ele será multiplicado.
- Assim, representamos infinitos valores com um conjunto finito de símbolos.

Formalizando

Para definir um sistema posicional, precisamos de:

1. **Uma Base (B):** O número de símbolos distintos.
2. **Conjunto de Símbolos:** Valores inteiros de 0 até $B - 1$.

Representação Genérica em Base B

Com a base (B) e o conjunto de símbolos definidos, temos:

- **Sequência de dígitos:** $s = b_i$, para $i \in -k \dots n$.
- **Cálculo do Valor:**

Fórmula Geral

$$\text{Valor} = \sum_{i=-k}^n b_i B^i$$

- **Expansão da Soma:**

$$b_n B^n + b_{n-1} B^{n-1} + \dots + b_0 B^0 + b_{-1} B^{-1} + \dots + b_{-k} B^{-k}$$

- **Convenção de Escrita:** A posição menos significativa fica sempre à direita:

$$b_n b_{n-1} \dots b_1 b_0, b_{-1} \dots b_{-k+1} b_{-k}$$

Exemplo de Decomposição: Inteiros (Base 10)

Considere a sequência **347** na base 10. Aplicando a fórmula da soma de produtos pelo peso da posição:

- **Decomposição:**

$$3 \times 10^2 + 4 \times 10^1 + 7 \times 10^0$$

$$3 \times 100 + 4 \times 10 + 7 \times 1 = 300 + 40 + 7$$

- **Resultado:**

$$347_{10}$$

Exemplo de Decomposição: Fracionários (Base 10)

Considere a sequência **58,25** na base 10. Note o uso de potências negativas para os dígitos após a vírgula:

- **Decomposição:**

$$5 \times 10^1 + 8 \times 10^0 + 2 \times 10^{-1} + 5 \times 10^{-2}$$

$$50 + 8 + \frac{2}{10} + \frac{5}{100} = 50 + 8 + 0,2 + 0,05$$

- **Resultado:**

$$58,25_{10}$$

Valor Posicional e Potências de 10

- **A Vírgula Decimal:** Atua como o separador entre a parte **inteira** (potências positivas) e a parte **fracionária** (potências negativas).
- **Estrutura de Pesos:** Cada posição possui um peso que é uma potência de 10 (10^n).

Decomposição do número 2745,214

$$(2 \times 10^3) + (7 \times 10^2) + (4 \times 10^1) + (5 \times 10^0) + (2 \times 10^{-1}) + (1 \times 10^{-2}) + (4 \times 10^{-3})$$

- **Regra Geral:** Qualquer número é a soma dos produtos de cada dígito pelo seu valor posicional.

Exemplo

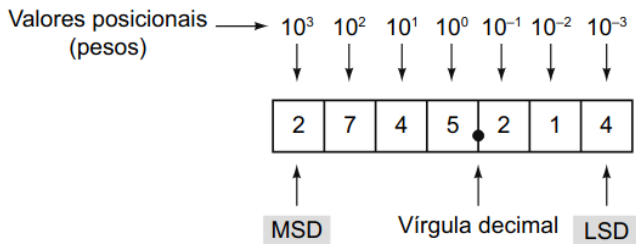


FIGURA 1.8 Valores posicionais de um número decimal expresso como potências de 10.

Decomponha os seguintes números decimais, considerando a notação posicional.

- a) **3014**
- b) **-25,5**
- c) **10,01**
- d) **-10750**

Exercício

Decomponha os seguintes números decimais, considerando a notação posicional.

a) **3014**

- $(3 \times 10^3) + (0 \times 10^2) + (1 \times 10^1) + (4 \times 10^0)$

b) **-25,5**

- $-((2 \times 10^1) + (5 \times 10^0) + (5 \times 10^{-1}))$

c) **10,01**

- $(1 \times 10^1) + (0 \times 10^0) + (0 \times 10^{-1}) + (1 \times 10^{-2})$

d) **-10750**

- $-((1 \times 10^4) + (0 \times 10^3) + (7 \times 10^2) + (5 \times 10^1) + (0 \times 10^0))$

Nota: O sinal é uma informação adicional à esquerda e não segue a convenção de posição/expoente.

Aula 2 - Utilizando outras bases

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 14, 2026

Comparativo de Bases e Símbolos

- Seguindo a regra do intervalo $[0, B - 1]$, temos os seguintes conjuntos:

Sistema	Base (B)	Símbolos Disponíveis	Intervalo
Binário	2	0, 1	0 a $B - 1$
Octal	8	0, 1, 2, 3, 4, 5, 6, 7	0 a $B - 1$
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	0 a $B - 1$
Hexadecimal	16	0-9, A, B, C, D, E, F	0 a $B - 1$

Convenção para Bases Maiores ($B > 10$)

Como não existem algarismos únicos para valores acima de 9, usamos letras: **A=10**, **B=11**, **C=12**, **D=13**, **E=14**, **F=15**

Bases de Numeração: Base 10 (Decimal)

- **Uso:** Base com a qual estamos familiarizados e interagimos como usuários.
- **Notação:**
 - Indicada pelo índice $_{10}$.
 - **Padrão (Default):** Se não houver indicação de base, assume-se que o número é decimal.

Exemplos

345 ou 345_{10}

Bases de Numeração: Base 2 (Binária)

- **Uso:** Linguagem nativa dos circuitos digitais.
- **Notação Acadêmica:** Indicada pelo índice $_2$.
- **Em Programação:** Precedida pelos caracteres 0B ou 0b.

Exemplos

10111₂ ou 0b10111

Bases de Numeração: Base 8 (Octal)

- **Notação Acadêmica:** Indicada pelo índice $_8$.
- **Em Programação:**
 - Precedida por 00 ou 0o.
 - Em algumas linguagens (como C antigo), precedida apenas por 0.

Exemplos

107_8 ou 0o107 ou 0107

Bases de Numeração: Base 16 (Hexadecimal)

- **Uso:** Representação compacta de binários.
- **Notação Acadêmica:** Indicada pelo índice $_{16}$.
- **Em Programação:** Precedida pelos caracteres 0X ou 0x.

Exemplos

$1C7_{16}$ ou `0x1C7`

Lembrete

Utiliza dígitos de 0 a 9 e as letras de A a F (para representar de 10 a 15).

Valores de 0 a 15 nas bases 2, 8, 10 e 16

10	2	8	16
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Exercício: Contagem em Múltiplas Bases

Escreva os números de 0 a 20 nas seguintes bases:
2, 5, 8, 10, 11 e 16.

Dicas!

- Na **Base 5**, o número após o 4 é o 10_5 .
- Na **Base 11**, o número 10 é representado pela letra *A*.

Gabarito: Contagem de 0 a 20 em Diferentes Bases

Base 10	Base 2	Base 5	Base 8	Base 11	Base 16
0	0	0	0	0	0
1	1	1	1	1	1
2	10	2	2	2	2
3	11	3	3	3	3
4	100	4	4	4	4
5	101	10	5	5	5
6	110	11	6	6	6
7	111	12	7	7	7
8	1000	13	10	8	8
9	1001	14	11	9	9
10	1010	20	12	A	A
11	1011	21	13	10	B
12	1100	22	14	11	C
13	1101	23	15	12	D
14	1110	24	16	13	E
15	1111	30	17	14	F
16	10000	31	20	15	10
17	10001	32	21	16	11
18	10010	33	22	17	12
19	10011	34	23	18	13
20	10100	40	24	19	14

Aula 3 - Conversão de bases

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 14, 2026

Conversão de uma base qualquer para base 10

Para converter um número de qualquer base B para a base 10, aplicamos a definição da notação posicional, realizando os cálculos na base 10:

Fórmula Única

$$\text{Valor}_{10} = \sum_{i=-k}^n b_i B^i$$

- **Passo a passo:**

1. Identifique o peso (potência de B) de cada posição.
2. Multiplique o dígito pelo seu peso.
3. Some todos os resultados.

Exemplos - Conversão de Binário e Octal para a base 10

- **Converter de Binário ($B = 2$) para decimal:** $10100,1_2$

$$(1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (0 \times 2^0) + (1 \times 2^{-1})$$

$$16 + 0 + 4 + 0 + 0 + 0,5 = \mathbf{20,5}_{10}$$

- **Converter de Octal ($B = 8$) para decimal:** 253_8

$$(2 \times 8^2) + (5 \times 8^1) + (3 \times 8^0)$$

$$(2 \times 64) + (5 \times 8) + (3 \times 1)$$

$$128 + 40 + 3 = \mathbf{171}_{10}$$

Exemplos de Conversão: Hexadecimal

- **Converter de Hexadecimal ($B = 16$) para decimal:** $C3_{16}$

Lembrete Importante

No sistema hexadecimal, letras devem ser substituídas pelos seus valores decimais antes do cálculo: **C = 12**.

$$(C \times 16^1) + (3 \times 16^0)$$

$$(12 \times 16) + (3 \times 1)$$

$$192 + 3 = \mathbf{195}_{10}$$

Exercícios: Conversão para Base 10

Converta os seguintes números para a base 10, demonstrando a decomposição posicional:

a) $11011,01_2$

b) 702_8

c) BF_{16}

Dica: Comece identificando os expoentes da base a partir da vírgula (para a esquerda 0, 1, 2... e para a direita -1, -2...).

Resposta $11011,01_2$ para decimal

$$\begin{aligned} & (11011,01)_2 \\ & 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} \\ & 16 + 8 + 0 + 2 + 1 + 0 + 0,25 \\ & (27,25)_{10} \end{aligned}$$

$$11011,01_2 = 27,25_{10}$$

Resposta 702₈ para decimal

$$(702)_8$$

$$7 \times 8^2 + 0 \times 8^1 + 2 \times 8^0$$

$$7 \times 64 + 0 + 2$$

$$448 + 2$$

$$(450)_{10}$$

$$702_8 = 450_{10}$$

Resposta BF_{16} para decimal

$$\begin{aligned} & (BF)_{16} \\ & B \times 16^1 + F \times 16^0 \\ & 11 \times 16 + 15 \\ & 176 + 15 \\ & (191)_{10} \end{aligned}$$

$$BF_{16} = 191_{10}$$

Conversão de Decimal para uma base qualquer

- **O Método:** Divisões Sucessivas pela base de destino.
- **Propriedade 1:** O resto da primeira divisão é o dígito de menor peso (b_0).
- **Propriedade 2:** Ao dividir o quociente sucessivamente, obtemos os dígitos seguintes ($b_1, b_2, \dots$).
- **Encerramento:** O processo termina quando o **quociente é zero**.

Formando o número

O resultado é formado pela leitura dos **restos** na ordem **inversa** (do último para o primeiro).

Exemplo Passo a Passo: 9_{10} para Binário

Vamos converter o número 9 para a base 2:

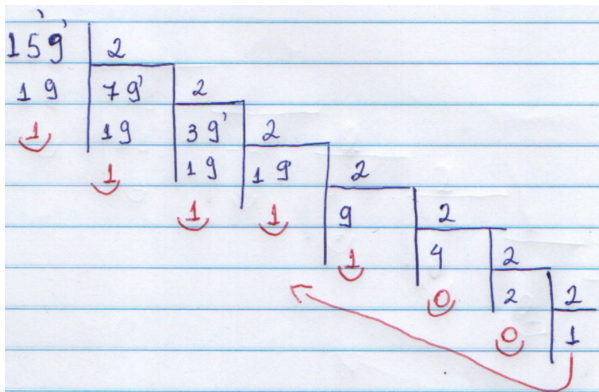
1. $9 \div 2 = 4$ com **resto 1** $\rightarrow (b_0)$
2. $4 \div 2 = 2$ com **resto 0** $\rightarrow (b_1)$
3. $2 \div 2 = 1$ com **resto 0** $\rightarrow (b_2)$
4. $1 \div 2 = 0$ com **resto 1** $\rightarrow (b_3)$

Resultado

Lendo os restos do último para o primeiro: **1001_2**

$$9 = 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

Exemplo do método das divisões sucessivas



$$159_{10} = 10011111_2$$

Exemplo do método das divisões sucessivas

38		2				
18		19	2			
0		9	2			
		1	4	2		
			0	2	2	
				0	2	1

$$38_{10} = 100110_2$$

Conversão: 352_{10} para Octal

Utilizar uma tabela ajuda a organizar o processo para números maiores:

Índice	Dividendo	Divisor	Quociente	Resto
0	352	8	44	0
1	44	8	5	4
2	5	8	0	5

- Lendo os restos de baixo para cima: **5, 4, 0**.
- **Resultado:** $352_{10} = 540_8$

Exemplo do método das divisões sucessivas

$(352)_{10}$

352	8		
32	44	8	
0	4	5	8
		5	0

$(540)_8$

The diagram illustrates the successive division method for converting the decimal number 352 to base 8. It shows a series of divisions where the quotient of one step becomes the dividend for the next. The remainders, written in red, are 0, 4, 5, and 5. These remainders are read from bottom to top to form the base-8 number 540. A red arrow points from the final remainder 5 to the first remainder 0, indicating the reading order.

$$352_{10} = 540_8$$

Conversão de 352_{10} para Binário

Índice	Dividendo	Divisor	Quociente	Resto
0	352	2	176	0
1	176	2	88	0
2	88	2	44	0
3	44	2	22	0
4	22	2	11	0
5	11	2	5	1
6	5	2	2	1
7	2	2	1	0
8	1	2	0	1

Lendo os restos do índice 8 ao 0: 101100000_2

Escreva sua **idade** e o **ano atual** nas seguintes bases, utilizando o método das divisões sucessivas:

- a) Base 11
- b) Base 2
- c) Base 8
- d) Base 16

Casos especiais onde uma base é potência da outra

- Para converter de uma base para outra qualquer, sempre é possível converter para a base 10 e em seguida da base 10 para a base final.
- No entanto, em alguns casos (quando uma base é uma potência da outra) pode-se evitar esse trabalho.
- As técnicas para as bases mais usadas em computação (2, 8 e 16) são exemplificadas a seguir.

Conversão de números em base 8 para base 2

- A base octal (8) é uma potência da base binária (2): $8 = 2^3$.
- Isso permite uma conversão direta e simplificada.
- Vamos desenvolver o raciocínio por trás do método.

Passo 1: Expansão posicional - Reescrevendo na base 2

Considere o número 253_8 :

$$\begin{aligned} 253_8 &= 3 \times 8^0 + 5 \times 8^1 + 2 \times 8^2 \\ &= 3 \times (2^3)^0 + 5 \times (2^3)^1 + 2 \times (2^3)^2 \\ &= 3 \times 2^{(3 \times 0)} + 5 \times 2^{3 \times 1} + 2 \times 2^{3 \times 2} \\ &= 3 \times 2^0 + 5 \times 2^3 + 2 \times 2^6 \end{aligned}$$

Passo 2: Tentativa inicial

- Temos os coeficientes 3, 5 e 2 multiplicando potências de 2.
- Seria tentador escrever 2005003_2 , colocando cada dígito em sua posição (0, 3 e 6).
- **Problema:** 2, 5 e 3 **não são dígitos válidos** na base binária (que só aceita 0 e 1).
- Precisamos traduzir esses valores (3, 5 e 2) para sua representação binária.

ATENÇÃO, nessa hora tome cuidado com mistura de bases

Passo 3: Decomposição dos coeficientes

Transformando 3, 5 e 2 em somas de potências de 2

$$3 = 1 + 2 = 1 \times 2^0 + 1 \times 2^1$$

$$5 = 1 + 4 = 1 \times 2^0 + 1 \times 2^2$$

$$2 = 0 + 2 = 1 \times 2^1$$

Passo 4: Substituição e desenvolvimento

$$\begin{aligned}253_8 &= 3 \times 2^0 + 5 \times 2^3 + 2 \times 2^6 \\&= (1 \times 2^0 + 1 \times 2^1) \times 2^0 + \\&\quad (1 \times 2^0 + 1 \times 2^2) \times 2^3 + \\&\quad (1 \times 2^1) \times 2^6\end{aligned}$$

Passo 5: Simplificação

Distribuindo e somando os expoentes:

$$\begin{aligned} 253_8 &= (1 \times 2^0) + (1 \times 2^1) + \\ &\quad (1 \times 2^3) + (1 \times 2^5) + \\ &\quad (1 \times 2^7) \\ &= 2^0 + 2^1 + 2^3 + 2^5 + 2^7 \end{aligned}$$

Passo 6: O número binário resultante

A soma das potências de 2 nos dá a representação binária final:

$$2^0 + 2^1 + 2^3 + 2^5 + 2^7 = 10101011_2$$

Ou seja, $253_8 = 10101011_2$.

Passo-a-passo feito a mão

$$(253)_8 = 2 \times 8^2 + 5 \times 8^1 + 3 \times 8^0$$

$$2 \times (2^3)^2 + 5 \times (2^3)^1 + 3 \times (2^3)^0$$

$$(1 \times 2^4) \times (2^6) + (2^2 + 1) \times (2^3) + (2^1 + 1) \times 2^0$$

$$2^7 + 2^5 + 2^3 + 2^1 + 2^0$$

Completo

as

posições

$$\rightarrow 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$= 10101011$$

Tabela

$$2^3 = 8$$

$$2 = 1 \times 2^1$$

$$3 = 2 + 1$$

$$5 = 2^2 + 1$$

Consequência do resultado anterior

O desenvolvimento anterior justifica o método que vamos realmente aplicar:

Como fazer?

Cada símbolo da representação **octal** deve ser expandido em sua representação com **3 bits**.

- Devem-se usar **zeros à esquerda** para completar os 3 bits em símbolos menores que 4 (100_2).

Exemplo: Sabendo que $2_8 = 010_2$, $5_8 = 101_2$ e $3_8 = 011_2$:

- $253_8 = \underbrace{010}_2 \text{ concat. com } \underbrace{101}_5 \text{ concat. com } \underbrace{011}_3$

Exemplo fazendo a concatenação

Ao realizar a junção dos grupos de 3 bits, temos:

Conversão Final

$$253_8 = 010101011_2$$

- $253_8 = 10101011_2$

Observação Importantíssima

O zero mais à esquerda (010...) pode ser **omitido**, uma vez que a concatenação foi finalizada e ele não altera o valor do número.

Tabela de Conversão: Octal $\rightarrow$ Binário

- Como $8 = 2^3$, cada dígito octal é **3 bits** na representação binária.
- Tabela de Substituição dígito octal por binário de 3 bits.

Digito Octal	3 dígitos em Binário
0_8	000_2
1_8	001_2
2_8	010_2
3_8	011_2
4_8	100_2
5_8	101_2
6_8	110_2
7_8	111_2

Exemplo: Conversão Binário → Octal

Convertendo 11100_2 para octal - Método Prático

- Para converter binário em octal, agrupamos os bits de **3 em 3** a partir da direita.
- $11100_2 = \underbrace{11}_{\text{grupo 1}} \underbrace{100}_{\text{grupo 2}}$
- Completamos com zero à esquerda se necessário: 011 100
- Cada grupo de 3 bits é convertido independentemente:

Grupo	Bits	Octal
Grupo 1 (esquerda)	011_2	3_8
Grupo 2 (direita)	100_2	4_8

- Concatenando os dígitos octais: 3 e 4
- Portanto: $11100_2 = 34_8$

Exemplo: Conversão Binário $\rightarrow$ Octal (Visualização)

4	3	2	1	0	índice
1	1	1	0	0	dígito

3 4

$$\begin{aligned}11100_2 &= 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 \\&= 2^4 + 2^3 + 2^2 \\&= (2^1 + 2^0) \times 2^3 + 2^2 \\&= (2 + 1) \times 8 + 4 \\&= 3 \times 8 + 4 \\&= 3 \times 8^1 + 4 \times 8^0 \\&= 34_8\end{aligned}$$

Conversão: Base 16 para Base 2

Como $16 = 2^4$, cada símbolo da representação hexadecimal corresponde a um grupo de **4 bits**.

Procedimento:

- Cada dígito hexadecimal deve ser expandido em sua representação binária de 4 bits.
- **Atenção:** Deve-se usar zeros à esquerda para completar os 4 bits em valores menores que 8 (ex: $1_{16} \rightarrow 0001_2$).

Exemplo Detalhado: $1C_{16}$

Vamos decompor o número $1C_{16}$ utilizando potências de 2:

- $1C_{16} = C \times 16^0 + 1 \times 16^1$
- $1C_{16} = 12 \times 16^0 + 1 \times 16^1$
- $1C_{16} = 12 \times (2^4)^0 + 1 \times (2^4)^1$
- $1C_{16} = 12 \times 2^0 + 1 \times 2^4$

Sabendo que $C_{16} = 12_{10} = 1100_2$ (ou $2^3 + 2^2$), substituímos:

- $1C_{16} = (2^3 + 2^2) \times 2^0 + 1 \times 2^4$
- $1C_{16} = 2^3 + 2^2 + 2^4$
- $1C_{16} = 11100_2$

Tabela de Conversão: Hexadecimal $\rightarrow$ Binário

- Como $16 = 2^4$, cada dígito hexadecimal corresponde a **4 bits** na representação binária.
- Substituímos cada símbolo pelo seu equivalente binário de 4 dígitos.

Hex	4 bits Binário	Hex	4 bits Binário
0 ₁₆	0000 ₂	8 ₁₆	1000 ₂
1 ₁₆	0001 ₂	9 ₁₆	1001 ₂
2 ₁₆	0010 ₂	A ₁₆	1010 ₂
3 ₁₆	0011 ₂	B ₁₆	1011 ₂
4 ₁₆	0100 ₂	C ₁₆	1100 ₂
5 ₁₆	0101 ₂	D ₁₆	1101 ₂
6 ₁₆	0110 ₂	E ₁₆	1110 ₂
7 ₁₆	0111 ₂	F ₁₆	1111 ₂

Método Simplificado (Mapeamento Direto)

A forma mais rápida é a concatenação de grupos de 4 bits:

Hexadecimal	Binário (4 bits)
1_{16}	0001_2
C_{16}	1100_2

Resultado:

$$1C_{16} = \underbrace{0001}_1 \underbrace{1100}_C = 00011100_2$$

Removendo os zeros à esquerda:

$$1C_{16} = 11100_2$$

Exercício

Converta **sem passar para a base 10** os seguintes números da base 8 para a base 2:

- a) 20_8
- b) 7_8
- c) 407_8
- d) 2022_8

Converta **sem passar para a base 10** os seguintes números da base 16 para a base 2:

- a) 20_{16}
- b) 7_{16}
- c) 407_{16}
- d) 2022_{16}
- e) $ABCD_{16}$

Exercício – Gabarito

Converta **sem passar para a base 10** os seguintes números da base 8 para a base 2 **grupos de 3 bits**:

a) $20_8 = 010\ 000_2 = 10000_2$

b) $7_8 = 111_2$

c) $407_8 = 100\ 000\ 111_2$

d) $2022_8 = 010\ 000\ 010\ 010_2 = 10000010010_2$

Converta **sem passar para a base 10** os seguintes números da base 16 para a base 2 **grupos de 4 bits**:

a) $20_{16} = 0010\ 0000_2 = 100000_2$

b) $7_{16} = 0111_2 = 111_2$

c) $407_{16} = 0100\ 0000\ 0111_2 = 10000000111_2$

d) $2022_{16} = 0010\ 0000\ 0010\ 0010_2 = 10000000100010_2$

e) $ABCD_{16} = 1010\ 1011\ 1100\ 1101_2$

Aula 4 - Operações aritméticas em diferentes bases

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 14, 2026

- O procedimento para realizar operações aritméticas em qualquer base é aplicar a **tabuada da operação** na base correspondente.
- Na **base 10**, fazemos isso de forma automática devido ao treinamento precoce.
- Em outras bases (2, 8, 16), como não temos isso na nossa cabeça.

Como fazer soma, por exemplo:

1. Trabalhamos coluna por coluna (da direita para a esquerda).
2. Realizamos a soma ou subtração.
3. Se o valor exceder a base, ocorre o "**vai um**".

Adição Binária: A Tabuada de Bits

Para realizar a adição binária sem converter para a base 10, utilizamos a tabuada da operação. Por ser pequena, sua memorização é simples:

Operando 1	Operando 2	Soma	Vai um (Carry)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table: Tabuada de adição de bits

- Quando há um "vai um", a próxima coluna passa a ter 3 operandos.
- Aplicamos a soma em etapas: $(1 + 1) + 1 = 10_2 + 1 = 11_2$ (soma 1 e vai 1).

Exemplo de Adição Binária: $11011_2 + 1001_2$

Somamos da direita para a esquerda, respeitando as colunas:

$$\begin{array}{rcccccc} \text{(vai um)} & 1 & 1 & 0 & 1 & 1 \\ & & 1 & 1 & 0 & 1 & 1_2 \\ + & & & 1 & 0 & 0 & 1_2 \\ \hline & 1 & 0 & 0 & 1 & 0 & 0_2 \end{array}$$

Resultado: $11011_2 + 1001_2 = 100100_2$

Nota: em binário, $1+1=10$ (fica 0 e vai 1) e $1+1+1=11$ (fica 1 e vai 1).

Exercício – Operações na Base 2

Faça as seguintes operações na base 2 **sem converter** os números a serem somados para a base 10 (você pode converter depois de realizar o exercício para conferir o resultado!):

a) $1101_2 + 010_2$

b) $1010_2 + 1010_2$

c) $1111_2 + 1111_2$

d) $101_2 + 1011_2$

Exercício – Operações na Base 2

Gabarito - Faça as seguintes operações na base 2 **sem converter** os números a serem somados para a base 10 (você pode converter depois de realizar o exercício para conferir o resultado!):

a) $1101_2 + 0010_2 = 1111_2$

b) $1010_2 + 1010_2 = 10100_2$

c) $1111_2 + 1111_2 = 11110_2$

d) $0101_2 + 1011_2 = 10000_2$

Exemplo: $172_8 + 30_8$

$$\begin{array}{r} \text{(carry)} \quad 1 \\ 1 \quad 7 \quad 2_8 \\ + \quad \quad 3 \quad 0_8 \\ \hline 2 \quad 2 \quad 2_8 \end{array}$$

Raciocínio por coluna:

- **Col. 0:** $2 + 0 = 2_8$.
- **Col. 1:** $7 + 3 = 10_{10}$. Como $10 = (1 \times 8) + 2$, temos 12_8 . Fica 2 e **vai** 1.
- **Col. 2:** $1 \text{ (carry)} + 1 = 2_8$.

Exemplo de Adição Hexadecimal: $5D_{16} + 5A_{16}$

$$\begin{array}{r} \text{(carry)} \quad 1 \\ 5 \quad D \quad 2_{16} \\ + \quad \quad 5 \quad A_{16} \\ \hline 6 \quad 2 \quad C_{16} \end{array}$$

Detalhamento do raciocínio:

Coluna 0: $2_{16} + A_{16} = 2 + 10 = 12_{10}$. Convertendo para hexa: C_{16} .

Coluna 1: $D_{16} + 5_{16} = 13 + 5 = 18_{10}$.

Como $18 = (1 \times 16) + 2$, temos 12_{16} . Fica 2 e **vai** 1.

Coluna 2: $1 \text{ (carry)} + 5 = 6_{16}$.

Resultado: $5D_{16} + 5A_{16} = 62C_{16}$

Exercícios: Adição em Bases 8 e 16

Realize as seguintes operações **sem converter os números para a base 10**:

Base 8 (Octal)

- a) $5_8 + 6_8$
- b) $27_8 + 11_8$
- c) $256_8 + 525_8$

Base 16 (Hexadecimal)

- a) $503C_{16} + 8_{16}$
- b) $503C_{16} + 64_{16}$

Dica: Some coluna por coluna e lembre-se do "vai um" sempre que o valor atingir ou superar a base.

Resultados na Base 8:

a) $5_8 + 6_8 = 13_8$

b) $27_8 + 11_8 = 40_8$

c) $256_8 + 525_8 = 1003_8$

Resultados na Base 16:

a) $503C_{16} + 8_{16} = 5044_{16}$

b) $503C_{16} + 64_{16} = 50A0_{16}$

Subtração em Diferentes Bases

- **Subtração Simples:** Quando o dígito do minuendo (cima) é maior ou igual ao do subtraendo (baixo).
- **Subtração com Empréstimo:** Quando o dígito de cima é menor que o de baixo, precisamos "pedir emprestado" à coluna da esquerda.

A Regra do Empréstimo (Borrow)

1. O dígito da esquerda (posição $i + 1$) perde **1**.
2. O dígito atual (posição i) ganha o **valor da base** (2, 8 ou 16).

Minuendo	Subtraendo	Diferença	Vem um (Borrow)
0	0	0	0
0	1	1	1 (Empréstimo)
1	0	1	0
1	1	0	0

Table: Tabuada de subtração de bits

Exemplo 1: Subtração Binária Simples

Neste caso, todos os bits de cima são maiores ou iguais aos de baixo:

$$\begin{array}{r} 1 \ 1 \ 0 \ 1 \ 1_2 \\ - \quad 1 \ 0 \ 0 \ 1_2 \\ \hline \mathbf{1 \ 0 \ 0 \ 1 \ 0_2} \end{array}$$

Resultado: $11011_2 - 1001_2 = 10010_2$

Exemplo 2: Subtração com Empréstimo

Quando temos $0 - 1$, pedimos emprestado à esquerda. O 1 vira 0 e a coluna atual ganha 10_2 (que vale 2_{10}).

$$\begin{array}{rcccccc} \text{(emprestado)} & -1 & -1 & +10 & & \\ & 1 & 1 & 0 & 1 & 1_2 \\ - & & 1 & 1 & 0 & 1_2 \\ \hline & \mathbf{0} & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{0}_2 \end{array}$$

Raciocínio:

- **Col. 2:** $0 - 1$ não dá. Pede emprestado: o 1 da Col. 3 vira 0. A Col. 2 vira 10_2 . $10_2 - 1 = 1$.
- **Col. 3:** Era 1, virou 0. $0 - 1$ não dá. Pede à Col. 4. O 1 da Col. 4 vira 0. A Col. 3 vira 10_2 . $10_2 - 1 = 1$.

Resultado: $11011_2 - 1101_2 = 1110_2$

Exercícios: Subtração Binária

Realize as seguintes operações **sem converter os números para a base 10**.

a) $1101_2 - 100_2$

b) $1010_2 - 101_2$

c) $1000_2 - 111_2$

d) $10011_2 - 101_2$

Gabarito: Subtração Binária

a) $1101_2 - 0100_2 = 1001_2$

b) $1010_2 - 0101_2 = 101_2$

c) $1000_2 - 0111_2 = 1_2$

d) $10011_2 - 00101_2 = 1110_2$

Dica: Para conferir, some o resultado com o subtraendo!

Subtração Octal

Diferente da base 10, o empréstimo aqui vale **8**.

Sem Empréstimo

$$\begin{array}{r} 1 \ 7 \ 2_8 \\ - \quad 3 \ 0_8 \\ \hline 1 \ 4 \ 2_8 \end{array}$$

Sem empréstimo, a sequência de símbolos é idêntica à base 10.

Com Empréstimo

$$\begin{array}{r} \text{(empréstado)} \quad -1 \quad +10_8 \\ \quad \quad \quad 1 \quad 2 \quad 2_8 \\ - \quad \quad \quad 7 \quad 0_8 \\ \hline \quad \quad \quad 0 \quad 3 \quad 2_8 \end{array}$$

Raciocínio Col. 1:

$2_8 < 7_8 \rightarrow$ pede empréstado.

$(2 + 8)_{10} = 10_{10}$.

$10 - 7 = 3$. Resultado: 3_8 .

Subtração Hexadecimal: Exemplo $5D2_{16} - 5A_{16}$

$$\begin{array}{r} \text{(emprestado)} \quad -1 \quad +10_{16} \\ 5 \quad D \quad 2_{16} \\ - 5 \quad A_{16} \\ \hline 5 \quad 7 \quad 8_{16} \end{array}$$

Detalhamento:

- **Col. 0:** $2 < A(10)$. Pede emprestado: $(2 + 16)_{10} = 18_{10}$.
 $18 - 10 = 8$. Resultado: 8_{16} .
- **Col. 1:** O $D(13)$ emprestou 1, virou $C(12)$.
 $12 - 5 = 7$. Resultado: 7_{16} .
- **Col. 2:** $5 - 0 = 5$.

Exercícios: Subtração Octal e Hexadecimal

Realize as operações diretamente na base indicada:

Base 8 (Octal)

a) $254_8 - 123_8$

b) $56_8 - 37_8$

Base 16 (Hexadecimal)

a) $503C_{16} - 40_{16}$

b) $50EA_{16} - 503C_{16}$

Dica: Converta o símbolo para decimal apenas para fazer a conta da coluna, depois volte para a base original.

Respostas Base 8:

a) $254_8 - 123_8 = 131_8$

b) $56_8 - 37_8 = 17_8$

Respostas Base 16:

a) $503C_{16} - 40_{16} = 4FFC_{16}$

b) $50EA_{16} - 503C_{16} = AE_{16}$

**No item (a) de Hexa, o 0 da posição 503C precisou pedir emprestado ao 5, tornando-se 15 (F) após passar o empréstimo adiante.*

Multiplicação em Qualquer Base

- O processo segue a mesma lógica da base 10: multiplicamos cada dígito do segundo operando por todos os dígitos do primeiro.
- Cada nova linha de produto parcial é deslocada uma posição à esquerda.
- O resultado final é a **soma** de todos os produtos parciais.

Multiplicação Binária

No mundo binário, a operação é simplificada:

- Se o multiplicador é **1**: Copiamos o primeiro operando.
- Se o multiplicador é **0**: A linha será composta apenas por zeros.

Multiplicação Binária: A Regra

A multiplicação binária é baseada em uma tabuada muito simples. Na prática, multiplicamos o primeiro operando bit a bit pelos dígitos do segundo operando.

Primeiro número	Segundo número	Produto
0	0	0
0	1	0
1	0	0
1	1	1

O que observar:

- Multiplicar por **1** é apenas repetir o número.
- Multiplicar por **0** resulta em zero.
- Lembre-se de deslocar cada linha de produto parcial à esquerda.

Exemplo de Multiplicação: $11011_2 \times 1001_2$

$$\begin{array}{r}
 11011 \\
 \times 1001 \\
 \hline
 11011 \\
 00000 \\
 00000 \\
 00000 \\
 \hline
 111011
 \end{array}$$

Dica: Após gerar as linhas de produto, aplique a regra da **soma binária** com atenção aos "vai um" (carries) gerados.

Exemplo Hexadecimal: $5D2_{16} \times 51_{16}$

$$\begin{array}{r} \\ \\ \\ \times \\ \hline \\ + 1 \\ \hline \mathbf{1} \mathbf{D} \mathbf{7} \mathbf{7} \mathbf{2} \end{array}$$

Destaque do cálculo (x5):

- $5 \times 2 = 10_{10} = \mathbf{A}_{16}$.
- $5 \times D(13) = 65_{10} \rightarrow (4 \times 16) + 1 = \mathbf{41}_{16}$. Fica **1** e vão **4**.
- $5 \times 5 = 25_{10}$. Com o carry: $25 + 4 = 29_{10} \rightarrow (1 \times 16) + 13 = \mathbf{1D}_{16}$.

Exercícios: Multiplicação em Bases Diversas

Realize as operações diretamente na base indicada:

Binário e Octal

- a) $1010_2 \times 11_2$
- b) $1100_2 \times 101_2$
- c) $123_8 \times 43_8$

Hexadecimal

- d) $1A2_{16} \times 21_{16}$

Respostas:

a) $1010_2 \times 11_2 = 11110_2$

b) $1100_2 \times 101_2 = 111100_2$

c) $123_8 \times 43_8 = 5531_8$

d) $1A2_{16} \times 21_{16} = 35E2_{16}$

Divisão Binária

- O algoritmo é idêntico ao da divisão decimal (método da chave).
- **Vantagem:** Cada bit do quociente só pode ser **0** ou **1**.
 - **1:** Se o dividendo parcial for maior ou igual ao divisor.
 - **0:** Se o dividendo parcial for menor que o divisor.
- É a operação base para implementações de hardware em camadas próximas ao processador.

Exemplo: $100100_2 \div 111_2$

1	0	0	1	0	0		1	1	1
	1	1	1				1	0	1
			0	1	0	0	0		
				1	1	1			
			0	0	0	1			

Figure: Processo de divisão bit a bit

Passo a Passo da Divisão: $100100_2 \div 111_2$

1. **Busca:** Percorremos o dividendo da esquerda para a direita até encontrar um valor $\geq$ ao divisor (111_2).
2. **Primeiro Bit:** Como $1001_2 \geq 111_2$, o primeiro bit do quociente é **1**.
3. **Subtração:** $1001_2 - 111_2 = 0010_2$ (resto parcial).
4. **Baixar Bit:** "Baixamos" o próximo 0. O novo dividendo parcial é 100_2 .
5. **Bit Zero:** Como $100_2 < 111_2$, o próximo bit do quociente é **0**.
6. **Novo Bit:** Baixamos o último 0. Agora temos 1000_2 .
7. **Bit Um:** Como $1000_2 \geq 111_2$, o bit final do quociente é **1**.
8. **Resto Final:** $1000_2 - 111_2 = 001_2$.

Resultado: Quociente = 101_2 — Resto = 1_2

Casos Particulares: Multiplicação e Divisão por B^n

Operações por potências da base (B^n) são simplificadas pelo **deslocamento da vírgula**.

- **Multiplicação ($\times B^n$):** Desloca-se a vírgula para a **direita** (acrescentando zeros se necessário).
- **Divisão ($\div B^n$):** Desloca-se a vírgula para a **esquerda**.

Analogia no Sistema Decimal

- $12 \times 10^1 = 120$ (acrescenta um 0)
- $120 \div 10^1 = 12$ (remove um 0 ou desloca a vírgula)

Essa lógica é idêntica para a base 2, substituindo 10 por 2.

Multiplicação por Potência de 2 (2^n)

Para multiplicar um binário por 2^n , basta acrescentar n zeros ao final do número.

Exemplos Práticos

- $11_2 \times 2^1$: $3_{10} \times 2 = 6_{10} \rightarrow \mathbf{110_2}$
- $101_2 \times 2^1$: $5_{10} \times 2 = 10_{10} \rightarrow \mathbf{1010_2}$

- **Exercício 18:** Realize as multiplicações na base 2:
 - a) $111 \times 2^1 = \dots$
 - b) $110 \times 2^2 = \dots$
 - c) $11 \times 2^3 = \dots$

Divisão por Potência de 2 (2^n)

Para dividir um binário por 2^n , deslocamos a vírgula n casas para a **esquerda**.

Exemplos Práticos

- $110_2 \div 2^1$: $6_{10} \div 2 = 3_{10} \rightarrow \mathbf{11}_2$
- $1000_2 \div 2^2$: $8_{10} \div 4 = 2_{10} \rightarrow \mathbf{10}_2$
- $111_2 \div 2^1$: $7_{10} \div 2 = 3,5_{10} \rightarrow \mathbf{11, 1}_2$

Prova Real ($11, 1_2$)

$$(1 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) = 2 + 1 + 0,5 = \mathbf{3,5}_{10}$$

Exercício 19: Divisão na Base 2

Efetue as divisões deslocando a vírgula conforme a potência de 2:

- a) $1010 \div 2^1$
- b) $1100 \div 2^2$
- c) $11000 \div 2^3$
- d) $101 \div 2^1$

Dica

Se o número original não tiver zeros suficientes para "cortar", a vírgula (que estava implícita à direita) deve aparecer e caminhar para a esquerda.

Tabela de Potências de 2 (2^0 a 2^{32})

n	2^n	Na Base 2
0	1	1
1	2	10
2	4	100
3	8	1000
4	16	10000
5	32	100000
6	64	1000000
7	128	10000000
8	256	100000000
9	512	1000000000
10	1.024	10000000000

n	2^n
11	2.048
12	4.096
13	8.192
14	16.384
15	32.768
16	65.536
17	131.072
18	262.144
19	524.288
20	1.048.576
21	2.097.152

n	2^n
22	4.194.304
23	8.388.608
24	16.777.216
25	33.554.432
26	67.108.864
27	134.217.728
28	268.435.456
29	536.870.912
30	1.073.741.824
31	2.147.483.648
32	4.294.967.296

Aula 5 - Precisão Finita

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 14, 2026

Números de Precisão Finita

- **No papel:** Usamos quantos dígitos forem necessários.
- **No Computador:** O espaço (memória) é limitado e definido no projeto do hardware.

Definição

Computadores lidam com uma quantidade **fixa** de dígitos. Isso nos força a trabalhar com um conjunto limitado de valores representáveis.

O que estudaremos nesta seção:

- Propriedades algébricas na precisão finita.
- Cálculo da quantidade de valores representáveis na base B .
- Representação de números inteiros com sinal.

Limitações da Precisão Finita

Considere um sistema hipotético que usa apenas **3 dígitos decimais** (000 a 999):

- **Valores impossíveis de representar:**
 - Maiores que 999 (Ex: 1000, 1500).
 - Menores que 0 (Ex: -1, -50).
 - Frações e números irracionais (Ex: 0,5 ou π).

Quebra de Propriedades

Diferente da matemática tradicional, os números de precisão finita **não possuem a propriedade de fechamento**.

A Propriedade de Fechamento

- **Na Matemática (Conjunto $\mathbb{Z}$):** Para qualquer i, j , os resultados de $i + j$, $i - j$ e $i \times j$ também são inteiros.
- **No Computador:** O resultado de uma operação pode "escapar" da faixa permitida.

Exemplos de Falha (Sistema de 3 dígitos)

- **Soma:** $900 + 500 = 1400$ (*Lupa: Cabe apenas 400?*)
- **Subtração:** $1 - 2 = -1$ (*Não há sinal negativo*)
- **Multiplicação:** $100 \times 10 = 1000$ (*Estourou o limite*)

OVERFLOW

- **O que é:** Ocorre quando o resultado de uma operação aritmética está fora da faixa de valores que o sistema consegue representar.
- **Analogia:** É como um hodômetro de carro que chega em 999.999km e volta para 000.000km.

Impacto no Software

Se o programador não prever o overflow, o computador pode produzir resultados bizarros (como somar dois números positivos e obter um número negativo ou um valor muito pequeno).

Propriedades Algébricas na Precisão Finita

Diferente da Matemática Clássica, onde os conjuntos numéricos são infinitos, o hardware precisa lidar com limites físicos. Isso afeta como as propriedades algébricas se comportam.

Detecção de Erro

No caso de **overflow**, o hardware (CPU) é projetado para identificar o estouro de capacidade e emitir um aviso (*flag*) para o sistema operacional.

Vamos analisar as três propriedades fundamentais:

1. **Comutatividade**
2. **Associatividade**
3. **Distributividade**

Comutatividade e Associatividade

- **Comutatividade:** Mantém-se inalterada.
 - $a + b = b + a$
 - $a \times b = b \times a$
 - A ordem dos operandos não altera o resultado, mesmo que ocorra overflow.
- **Associatividade:** É satisfeita, desde que o resultado final seja válido.
 - $(a + b) + c = a + (b + c)$
 - $(a \times b) \times c = a \times (b \times c)$

Associatividade

Se o resultado final estiver dentro da capacidade de representação, o valor será o mesmo. Se houver overflow, ele ocorrerá independentemente da ordem de execução, tornando o resultado inválido em ambos os casos.

Distributividade na Precisão Finita

A última propriedade fundamental é a **Distributividade** da multiplicação sobre a adição:

Fórmula

$$a \times (b + c) = (a \times b) + (a \times c)$$

- **No Computador:** A propriedade se mantém válida para números inteiros.
- **A Ressalva:** Assim como na associatividade, a igualdade é garantida se:
 1. O resultado final for **válido** (dentro da faixa representável).
 2. **OU** se ambos os caminhos de cálculo resultarem em um valor **inválido** (Overflow).

Conclusão das Propriedades

Para **Inteiros**, a Álgebra de Precisão Finita tenta mimetizar a Álgebra Clássica. O maior "vilão" não é a quebra das regras de ordem, mas sim o **estouro do limite de bits**.

Por que as Propriedades importam?

Mesmo sem decorar os nomes, usamos essas regras para simplificar cálculos no dia a dia:

- **Comutatividade + Associatividade:** Permitem somar uma lista de números em qualquer ordem (ex: agrupar números que somam 10 para facilitar a conta de cabeça).
- **Distributividade:** É a base para **colocar fatores em evidência** em expressões algébricas.

No Computador

Essas manipulações são usadas pelos **Compiladores** para otimizar o código, reorganizando as operações para que o processador as execute de forma mais rápida.

Quantos números cabem no sistema?

Para entender a capacidade de um hardware, precisamos resolver um problema de contagem:

O Problema

Dados B símbolos diferentes, de quantas maneiras podemos preencher n posições distintas (onde a ordem importa e símbolos podem se repetir)?

- **Posição 0:** B opções.
- **Posição 1:** B opções.
- ...
- **Posição $n - 1$:** B opções.

$$\text{Total de Combinações} = B^n$$

Princípio Multiplicativo: B^n

A fórmula B^n surge do princípio fundamental da contagem:

- Com **1 posição**: B possibilidades.
- Com **2 posições**: $B \times B = B^2$ possibilidades.
- Com **3 posições**: $B \times B \times B = B^3$ possibilidades.

Exemplo em Binário ($B = 2$)

Se um computador usa ****8 bits**** ($n = 8$):

$$2^8 = 256 \text{ combinações diferentes}$$

Se ele usa 10 bits:

$$2^{10} = 1.024 \text{ combinações}$$

Exercícios: Capacidade de Representação

Utilizando a fórmula B^n , determine a quantidade de combinações possíveis para cada caso:

- a) Quantos números diferentes podem ser expressos usando **2 dígitos** na base **10**?
- b) Quantos números diferentes podem ser expressos usando **5 dígitos** na base **2**?
- c) Quantos números diferentes podem ser expressos usando **32 dígitos** na base **2**?

Dica: Lembre-se da nossa tabela de potências de 2 vista anteriormente!

a) Base 10, $n = 2$:

$$10^2 = \mathbf{100} \text{ combinações}$$

(Os números de 00 a 99)

b) Base 2, $n = 5$:

$$2^5 = \mathbf{32} \text{ combinações}$$

(Os números binários de 00000 a 11111)

c) Base 2, $n = 32$:

$$2^{32} = \mathbf{4.294.967.296} \text{ combinações}$$

(Aproximadamente 4,3 bilhões de valores diferentes)

A quantidade de bits (n) define o "tamanho do universo" de números que podemos contar. Se o resultado de uma conta for o número $2^{32} + 1$, ele não "caberá" no universo que só temos 32 bits, gerando o **Overflow**.

Como representar o sinal?

Até agora, vimos números "sem sinal" (unsigned), onde todos os bits representam magnitude. Para incluir números negativos, precisamos de convenções:

- **Números sem sinal:** Faixa de 0 a $B^n - 1$.
- **Números com sinal:** Três esquemas principais:
 1. **Sinal-Magnitude:** O bit mais à esquerda indica o sinal ($0 = +, 1 = -$).
 2. **Excesso de 2^{n-1} :** O valor é deslocado por uma constante.
 3. **Complementos (1 e 2):** O bit de sinal faz parte do valor aritmético.

O problema do Zero

Como n bits geram uma quantidade par de combinações (2^n), ao usarmos uma para o zero, sobra um número ímpar de padrões para dividir entre positivos e negativos.

O Esquema Sinal-Magnitude

b_{n-1}	b_{n-2}	$\dots$	b_1	b_0
Sinal	Magnitude (Valor Absoluto)			

- **Bit de Sinal (b_{n-1}):** $0 \rightarrow$ Positivo (+)
- $1 \rightarrow$ Negativo (-).
- **Magnitude:** Representada pelos $n - 1$ bits restantes.
- **Faixa de Valores:** De $-(B^{n-1} - 1)$ até $B^{n-1} - 1$.

O Problema do "Zero Duplo"

Neste sistema, o zero possui duas representações:

- $+0$ (ex: 0000) e -0 (ex: 1000).

Isso desperdiça uma combinação e complica a lógica do hardware.

Representação Sinal-Magnitude ($n = 4$)

A tabela abaixo apresenta todas as 16 combinações possíveis para 4 bits:

Binário	Valor Decimal	Binário	Valor Decimal
0000	+0	1000	-0
0001	+1	1001	-1
0010	+2	1010	-2
0011	+3	1011	-3
0100	+4	1100	-4
0101	+5	1101	-5
0110	+6	1110	-6
0111	+7	1111	-7

- **Faixa de representação:** $[-7, +7]$.
- Observe a simetria: o bit mais significativo (MSB) apenas altera o sinal, mantendo a magnitude idêntica.

Exemplos e Exercício (Sinal-Magnitude)

Considere $n = 4$ bits:

- $0011_2 \rightarrow +3_{10}$
- $1011_2 \rightarrow -3_{10}$
- $1000_2 = 0000_2 = 0$

Converta os decimais para binário usando **5 bits** em **Sinal-Magnitude**:

- a) -15
- b) -1
- c) 0 (apresente as duas formas)
- d) 10

Gabarito: Exercício 21

Para $n = 5$ bits, o primeiro bit é o sinal e os outros 4 são a magnitude:

- a) $-15 \rightarrow$ Sinal: 1 $\rightarrow$ Magnitude (15_{10}): $1111_2 \rightarrow$ **11111**
- b) $-1 \rightarrow$ Sinal: 1 $\rightarrow$ Magnitude (1_{10}): $0001_2 \rightarrow$ **10001**
- c) $0 \rightarrow$ **00000** ($+0$) e **10000** (-0)
- d) $10 \rightarrow$ Sinal: 0 $\rightarrow$ Magnitude (10_{10}): $1010_2 \rightarrow$ **01010**

Reflexão

Note que em Sinal-Magnitude, para tornar um número negativo, basta "ligar" o bit mais significativo. O restante do número permanece idêntico.

O Esquema de Excesso de 2^{n-1}

Neste sistema, não "reservamos" um bit de sinal isolado. Em vez disso, somamos um valor fixo ao número real para deslocá-lo para a faixa positiva.

Funcionamento

Para n bits, o excesso é 2^{n-1} .

$$\text{Representação Binária} = \text{Valor Decimal} + 2^{n-1}$$

- **Inversão de Lógica:** Ao contrário do Sinal-Magnitude, aqui:
 - Números que começam com **0** são **Negativos**.
 - Números que começam com **1** são **Positivos** (ou zero).
- **Vantagem:** Mantém a **ordem crescente** dos bits igual à ordem dos valores decimais.
- **Zero Único:** O valor 0 é representado pelo padrão $100\dots 0_2$.

Exemplo: $n = 4$ (Excesso de $2^3 = 8$)

Com 4 bits, somamos 8 a cada valor para achar seu código:

Decimal	Cálculo (+8)	Binário
-8 (mínimo)	$-8 + 8 = 0$	0000
-7	$-7 + 8 = 1$	0001
0	$0 + 8 = 8$	1000
+1	$1 + 8 = 9$	1001
+7 (máximo)	$7 + 8 = 15$	1111

- **Faixa de Valores** ($n = 4$): de -8 a $+7$.
- **Regra Geral**: de $-(2^{n-1})$ a $2^{n-1} - 1$.

Tabela Completa: Excesso de $2^3 = 8$ ($n = 4$)

Binário	Cálculo	Decimal	Binário	Cálculo	Decimal
0000	$0 - 8$	-8	1000	$8 - 8$	0
0001	$1 - 8$	-7	1001	$9 - 8$	$+1$
0010	$2 - 8$	-6	1010	$10 - 8$	$+2$
0011	$3 - 8$	-5	1011	$11 - 8$	$+3$
0100	$4 - 8$	-4	1100	$12 - 8$	$+4$
0101	$5 - 8$	-3	1101	$13 - 8$	$+5$
0110	$6 - 8$	-2	1110	$14 - 8$	$+6$
0111	$7 - 8$	-1	1111	$15 - 8$	$+7$

- **MSB = 0:** Números negativos.
- **MSB = 1:** Números positivos e o zero.
- Esta representação é muito utilizada em expoentes de números em **Ponto Flutuante (IEEE 754)**.

Exercício 22: Excesso de 2^{n-1} ($n = 5$)

Para $n = 5$, o excesso é $2^{5-1} = 2^4 = \mathbf{16}$.

Regra: Some 16 ao número e converta o resultado para binário de 5 bits.

Converta para binário (5 bits, com Excesso de 2^{n-1}):

- a) -15
- b) -1
- c) 0
- d) 10

a) $-15: -15 + 16 = 1 \rightarrow 00001_2$

b) $-1: -1 + 16 = 15 \rightarrow 01111_2$

c) $0: 0 + 16 = 16 \rightarrow 10000_2$

d) $10: 10 + 16 = 26 \rightarrow 11010_2$

Dica Visual

Repare que o zero (10000) divide a tabela exatamente no meio. Qualquer número menor que 16 começará com 0 (negativos), e qualquer número ≥ 16 começará com 1 (positivos).

Representação em Excesso de 16 ($n = 5$)

Para $n = 5$, o excesso é $2^{5-1} = 16$. A faixa de valores vai de -16 a $+15$.

Binário	Decimal	Binário	Decimal	Binário	Decimal
00000	-16	01011	-5	10110	+6
00001	-15	01100	-4	10111	+7
00010	-14	01101	-3	11000	+8
00011	-13	01110	-2	11001	+9
00100	-12	01111	-1	11010	+10
00101	-11	10000	0	11011	+11
00110	-10	10001	+1	11100	+12
00111	-9	10010	+2	11101	+13
01000	-8	10011	+3	11110	+14
01001	-7	10100	+4	11111	+15
01010	-6	10101	+5	—	

- **Cálculo:** Valor = Inteiro sem sinal $- 16$.
- O padrão 10000 sempre representa o zero real.

Aula 6 - Precisão Finita (parte 2)

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
April 14, 2026

- **Estratégia:** Inverter todos os bits para negar o número ($0 \rightarrow 1$ e $1 \rightarrow 0$).
- **Bit de Sinal:** O bit mais à esquerda define o sinal (0 pos, 1 neg).
- Assim como no Sinal-Magnitude, o Complemento a 1 possui **duas representações para o zero**:
 - 0000 (+0) e 1111 (−0 em $n = 4$).
- **Antes do Complemento a 2, utilizava-se o Complemento a 1**
- O **Complemento a 2** elimina o "zero duplo" e é o padrão nos computadores desde 1965.

Complemento a 1 com $n = 4$

Observe como cada número negativo é exatamente a inversão bit a bit do seu equivalente positivo:

Decimal	Binário	Inversão	Binário	Decimal
+0	0000	$\longleftrightarrow$	1111	-0
+1	0001	$\longleftrightarrow$	1110	-1
+2	0010	$\longleftrightarrow$	1101	-2
+3	0011	$\longleftrightarrow$	1100	-3
+4	0100	$\longleftrightarrow$	1011	-4
+5	0101	$\longleftrightarrow$	1010	-5
+6	0110	$\longleftrightarrow$	1001	-6
+7	0111	$\longleftrightarrow$	1000	-7

- **Funcionamento:** Para negar, basta aplicar o operador NOT (inverter 0 e 1).
- **O Problema:** Note o +0 (0000) e o -0 (1111). Essa duplicidade gera desperdício de uma combinação e circuitos de teste mais complexos.

Para encontrar o negativo de um número ($x \rightarrow -x$) em complemento a 2 faça o seguinte:

1. **Complemento a 1:** Inverta os bits.
2. **Somar 1:** Adicione 1 ao resultado obtido.

Exemplo de complemento a 2 ($+3 \rightarrow -3$):

- $+3 = 0011_2$
- Inverte (complemento a 1) $\rightarrow 1100_2$
- Some 1 $\rightarrow 1101_2$ (que é -3)

Complemento a 2: A Lógica dos Pesos

Neste sistema, o bit mais significativo (MSB) tem **peso negativo**, enquanto os demais mantêm pesos positivos:

Cálculo para $n = 4$ bits

- $0101 = -0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = \mathbf{5}$
- $1011 = -1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = -8 + 3 = \mathbf{-5}$
- $1111 = -1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = -8 + 7 = \mathbf{-1}$
- $1000 = -1 \cdot 2^3 + 0 + 0 + 0 = \mathbf{-8}$ (*Mínimo*)

Vantagem: O zero é único (0000) e as operações aritméticas são simplificadas no hardware.

Tabela Completa: Complemento a 2 ($n = 4$)

Binário	Valor Decimal	Binário	Valor Decimal
0000	0	1000	-8
0001	+1	1001	-7
0010	+2	1010	-6
0011	+3	1011	-5
0100	+4	1100	-4
0101	+5	1101	-3
0110	+6	1110	-2
0111	+7	1111	-1

Faixa de Valores: de -2^{n-1} até $2^{n-1} - 1$ (no caso, -8 a $+7$).

Visualizacao circular Complemento a 2

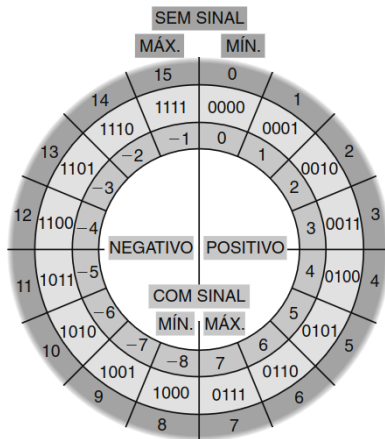


FIGURA 6.3 Um círculo numérico de quatro bits.

Números Positivos e o Zero em Complemento a 2

Para a faixa de 0 até $2^{n-1} - 1$, a representação em **Complemento a 2** é idêntica à de **Sinal-Magnitude**:

Regras de Representação

- **Bit de Sinal:** O bit mais significativo (MSB) é sempre **0**.
- **Magnitude:** Os demais bits seguem a notação posicional binária padrão.

Exemplo ($n = 4$)

O valor $+5_{10}$ é sempre 0101 em:

- Binário puro (4 bits)
- Sinal-Magnitude ($n = 4$)
- Complemento a 2 ($n = 4$)

Números Negativos em Complemento a 2

Para calcular a representação de um número negativo na faixa de -1 até -2^{n-1} , seguimos o algoritmo de três passos:

Algoritmo de Conversão ($x \rightarrow -x$)

1. **Magnitude:** Converte o valor absoluto do número para binário com n bits.
2. **Complemento a 1:** Inverte cada bit (0 vira 1, 1 vira 0).
3. **Soma 1** ao resultado anterior.
 - *Importante: Ignore o "vai-um" (carry-out) do bit mais significativo, se houver.*

Considere o descarte do carry: Este processo garante que a soma de um número com seu negativo resulte em zero.

Exemplos Práticos ($n = 4$)

Calculando o valor -3

- **Passo 1 ($|3|$):** 0011
- **Passo 2 (Comp. 1):** 1100
- **Passo 3 (Soma 1):** $1100 + 1 = 1101$
- **Resultado:** $-3_{10} = 1101_2$

O caso do Zero (0)

- **Passo 1 ($|0|$):** 0000
- **Passo 2 (Comp. 1):** 1111
- **Passo 3 (Soma 1):** $1111 + 1 = [1]0000$
- **Resultado:** 0000 (O "vai-um" é ignorado, mantendo o zero único).

Resumo de Comparação ($n = 4$)

Para o valor decimal 3:

- **Positivo (+3):** 0011
- **Negativo (-3):** 1101

$$\begin{array}{r} 0011 (+3) \\ + \quad 1101 (-3) \\ \hline [1] \quad 0000 (0) \end{array}$$

Dica de Hardware: Esse algoritmo é eficiente porque permite que o circuito de subtração seja basicamente um somador.

Faixa de Representação em Complemento a 2

Para uma palavra de n bits, a faixa de valores é assimétrica devido à representação única do zero:

- **Números Positivos e Zero:** De 0 até $2^{n-1} - 1$.
 - Temos $n - 1$ bits disponíveis para a magnitude.
- **Números Negativos:** De -1 até -2^{n-1} .
 - São 2^{n-1} combinações negativas.

Amplitude Total

A faixa completa vai de -2^{n-1} **até** $2^{n-1} - 1$.

- Total de valores: $2 \times 2^{n-1} = 2^n$ combinações distintas.
- **Vantagem:** Não há desperdício de combinação com o "zero duplo".

Exercício: Complemento a 2 com $n = 5$ bits

Escreva os números abaixo na base 2 usando a notação de **Complemento a 2**:

- a) -15
- b) -1
- c) 0
- d) 10

Respostas do Exercício ($n = 5$)

Confira os resultados para a representação de 5 bits:

Decimal	Binário (Complemento a 2)	Explicação
-15	10001	Inverte 01111 $\rightarrow$ 10000 +1
-1	11111	Inverte 00001 $\rightarrow$ 11110 +1
0	00000	Padrão único
10	01010	Positivo direto

Dica para os negativos: 1. Ache o positivo em 5 bits (ex:

15 = 01111).

2. Inverta os bits (10000).

3. Some 1 (10001).

Nota: Em $n = 5$, a faixa permitida é de -16 a $+15$. Todos os valores acima estão dentro do limite.

Exercício: Interpretando Complemento a 2 ($n = 7$)

Transcreva os números abaixo (em complemento a 2 de 7 bits) para a base 10.

a) 1111110

b) 1000001

c) 0111111

d) 1010101

Respostas do Exercício ($n = 7$)

Confira os valores correspondentes na base 10:

- a)** $1111110 \rightarrow \text{Inverte } (0000001) + 1 = 0000010_2 \rightarrow -2$
- b)** $1000001 \rightarrow \text{Inverte } (0111110) + 1 = 0111111_2 \rightarrow -63$
- c)** $0111111 \rightarrow \text{Começa com 0 (positivo)}$
 $\rightarrow 32 + 16 + 8 + 4 + 2 + 1 \rightarrow +63$
- d)** $1010101 \rightarrow \text{Inverte } (0101010) + 1 = 0101011_2 \rightarrow -43$

Se a sequência começa com 1, o número é negativo!

Negação (Inverso Aditivo) em Complemento a 2

"Negar" um número significa inverter o seu sinal ($x \rightarrow -x$).

- **No Sinal-Magnitude:** Basta inverter o bit mais significativo.
- **No Complemento a 2:** O processo de negação é **Complemento a 1 e Somar 1**.

Algoritmo

Independentemente de o número original ser positivo ou negativo, o processo para encontrar seu inverso aditivo é o mesmo:

1. Inverte-se todos os bits ($0 \leftrightarrow 1$).
2. Soma-se 1 ao resultado.

Exemplo ($n = 4$): De -3 para $+3$

- Partimos de -3 : 1101
- Invertemos os bits: 0010
- Somamos 1: $0010 + 1 = 0011$ (que é $+3$)

Exercício: Negação em 7 bits

Obtenha a representação da **negação** (inverso aditivo) de cada número abaixo, considerando a codificação Complemento a 2 com $n = 7$:

a) 1111110

b) 1000001

c) 0111111

d) 1010101

Lembre-se: O procedimento é o mesmo para todos: **Inverter + Somar 1.**

Respostas: Inverso Aditivo ($n = 7$)

a) 1111110 (-2)

- Inverte (0000001) $+1 \rightarrow$ **0000010** ($+2$)

b) 1000001 (-63)

- Inverte (0111110) $+1 \rightarrow$ **0111111** ($+63$)

c) 0111111 ($+63$)

- Inverte (1000000) $+1 \rightarrow$ **1000001** (-63)

d) 1010101 (-43)

- Inverte (0101010) $+1 \rightarrow$ **0101011** ($+43$)

Atenção

A negação de um número positivo **sempre** resultará em um número começando com 1, e vice-versa (exceto para o zero).

Adição em Complemento a 2

A grande vantagem do Complemento a 2 é que a adição é feita exatamente como nos números sem sinal, ignorando se o bit mais à esquerda representa sinal ou magnitude.

A Regra do Overflow

Diferente dos números sem sinal, o "vai-um" (carry-out) final não indica necessariamente um erro. No Complemento a 2, ocorre **overflow** apenas quando:

- Somamos dois números de **mesmo sinal** e o resultado tem **sinal oposto**.
- **Positivo + Negativo**: Nunca gera overflow.
- **Carry-out**: O vai-um que "sobra" à esquerda do bit de sinal deve ser **descartado**.

Exemplos de Adição ($n = 8$)

1. Positivos (sem overflow):

$$\begin{array}{r} \textcolor{red}{11} \text{ } \textcolor{red}{11} \\ 00011011_2 (27) \\ + 01001001_2 (73) \\ \hline 01100100_2 (100) \end{array}$$

2. Negativos (sem overflow):

$$\begin{array}{r} \textcolor{red}{11111} \text{ } \textcolor{red}{11} \\ 11111011_2 (-5) \\ + 11001001_2 (-55) \\ \hline [1]11000100_2 (-60) \end{array}$$

3. Sinais opostos:

$$\begin{array}{r} \textcolor{red}{11} \text{ } \textcolor{red}{11} \\ 11001001_2 (-55) \\ + 00011011_2 (27) \\ \hline 11100100_2 (-28) \end{array}$$

4. Negativos com Overflow:

$$\begin{array}{r} \textcolor{red}{1} \text{ } \textcolor{red}{11} \text{ } \textcolor{red}{11} \\ 10011011_2 (-99) \\ + 11001001_2 (-60) \\ \hline [1]01100100_2 !! \end{array}$$

Erro: Negativo + Negativo deu Positivo.

Subtração em Complemento a 2

A subtração é simplificada transformando-a em uma adição com o inverso aditivo:

$$A - B \implies A + (-B)$$

Procedimento

1. Mantenha o primeiro operando (*op1*).
2. Calcule o **Complemento a 2** do segundo operando (*op2*).
3. Some os dois valores seguindo as regras da adição.

Isso permite que o hardware use o mesmo circuito somador para as duas operações.

Exercício: Aritmética com 4 bits

Realize as operações usando **Complemento a 2 com 4 bits**. Indique se houve overflow.

a) $5 - 3$

b) $2 - 4$

c) $(-5) + (-1)$

d) $4 + 2$

d) $7 + 3$

d) $(-4) + (-5)$

Lembre-se da faixa de 4 bits: $[-8, +7]$.

Gabarito: Aritmética com 4 bits

- a) $0101 + 1101 = [1]0010 (+2)$ OK
- b) $0010 + 1100 = 1110 (-2)$ OK
- c) $1011 + 1111 = [1]1010 (-6)$ OK
- d) $0100 + 0010 = 0110 (+6)$ OK
- e) $0111 + 0011 = \mathbf{1010}$ OVERFLOW!
• *Positivo + Positivo resultou em negativo.*
- f) $1100 + 1011 = [1]\mathbf{0111}$ OVERFLOW!
• *Negativo + Negativo resultou em positivo.*

Multiplicação e Divisão em Complemento a 2

Diferente da adição, a multiplicação e a divisão no Complemento a 2 geralmente seguem uma estratégia de "valor absoluto":

Algoritmo Geral

1. **Preparação:** Se um operando for negativo, calcula-se seu Complemento a 2 para obter o valor absoluto ($|x|$).
2. **Operação:** Multiplica-se ou divide-se os valores absolutos como números sem sinal.
3. **Sinal do Resultado:**
 - Sinais iguais $\rightarrow$ Resultado Positivo.
 - Sinais opostos $\rightarrow$ Calcula-se o Complemento a 2 do resultado para torná-lo negativo.

Overflow na Multiplicação

Em uma representação de n bits, ocorre overflow se o produto dos valores absolutos atingir o bit de sinal ($n - 1$) ou além.

Exemplos de Multiplicação ($n = 8$)

1. Positivos com Overflow: $27 \times 5 = 135$ (Limite é 127)

	0	0	0	1	1	0	1	1_2
$\times$	0	0	0	0	0	1	0	1_2
	1!!	1	1	1				
	0	0	0	1	1	0	1	1
+	0	1	1	0	1	1		
	1!!	0	0	0	0	1	1	1

2. Positivos sem Overflow: $11 \times 5 = 55$

	0	0	0	0	1	0	1	1_2
$\times$	0	0	0	0	0	1	0	1_2
				1				
	0	0	0	0	1	0	1	1
+	0	0	1	0	1	1		
	0	0	1	1	0	1	1	1

Exemplo: Multip. de positivo e um negativo com *overflow*

Considere: 10001011_2 (-117) $\times$ 00000101_2 (5) em $n = 8$.

Passo 1: Obter valor absoluto de -117 :

	1	0	0	0	1	0	1	1_2
inv	0	1	1	1	0	1	0	0_2
+								1
	0	1	1	1	0	1	0	1_2

Passo 2: Multiplicar 117×5 :

	0	1	1	1	0	1	0	1_2
$\times$	0	0	0	0	0	1	0	1_2
	1	1	1	1		1		
	0	1	1	1	0	1	0	1
+ 1!!	1!!	1	0	1	0	1		
... !!	0	1	0	0	1	0	0	1

Expansão de Bits (Extensão de Sinal)

Frequentemente, precisamos converter um número de n bits para uma representação de m bits ($m > n$) mantendo seu valor original.

A Regra da Extensão de Sinal

Para expandir um número em **Complemento a 2**:

1. Move-se o bit de sinal para a nova posição mais à esquerda.
 2. Preenchem-se as novas posições com o **mesmo valor do bit de sinal**.
- Se o número é **positivo** (MSB=0), preenchemos com **0s** à esquerda.
 - Se o número é **negativo** (MSB=1), preenchemos com **1s** à esquerda.

Exemplos de Expansão ($4 \rightarrow 8$ bits)

Exemplo 1: Valor Positivo (+4)

- Representação 4 bits: 0100
- Extensão para 8 bits: 00000100

Exemplo 2: Valor Negativo (-4)

- Representação 4 bits: 1100
- Extensão para 8 bits: 11111100

Por que funciona? No caso do -4:

- 4 bits: $-8 + 4 = -4$
- 8 bits: $-128 + 64 + 32 + 16 + 8 + 4 = -128 + 124 = -4$

Exercício: Extensão de Sinal e Hexadecimal

Mostre que os valores hexadecimais abaixo, em notação de **Complemento a 2**, representam o valor -5_{10} :

- a) B ($n = 4$ bits)
- b) FB ($n = 8$ bits)
- c) FFB ($n = 12$ bits)

Dica de Resolução

Converta o dígito mais significativo para binário e verifique o bit de sinal.

- B em binário é 1011.
- Como o MSB é **1**, o número é negativo.
- Aplique o processo inverso (Inverter + Somar 1) para conferir a magnitude.

Resolução: A consistência do -5

a) $B \rightarrow 1011$.

- Inverso: $0100 + 1 = 0101$ (5_{10}). Logo: -5 .

b) $FB \rightarrow 1111\ 1011$.

- Inverso: $0000\ 0100 + 1 = 0000\ 0101$ (5_{10}). Logo: -5 .

c) $FFB \rightarrow 1111\ 1111\ 1011$.

- Inverso: $0000\ 0000\ 0100 + 1 = 0000\ 0101$ (5_{10}). Logo: -5 .

Conclusão

Note que FB e FFB são apenas extensões de sinal de B. Em hexadecimal, se o número for negativo, ele será preenchido com F (1111_2) à esquerda.

O caminho inverso (reduzir o número de bits) nem sempre preserva o valor. Ao truncar de n para k bits ($k < n$), eliminamos os $n - k$ bits mais significativos.

- **Números sem sinal:** O truncamento equivale a $x \bmod 2^k$.

Exemplo: Truncando 1100_2 (12_{10})

- **Para 3 bits:** $1|100 \rightarrow 100_2$ (4_{10}).
 - Cálculo: $12 \bmod 2^3 = 12 \bmod 8 = 4$.
- **Para 2 bits:** $11|00 \rightarrow 00_2$ (0_{10}).
 - Cálculo: $12 \bmod 2^2 = 12 \bmod 4 = 0$.

Atenção: No Complemento a 2, o truncamento pode alterar drasticamente o valor e até o **sinal** do número!